CareOps

Smart Healthcare Operations & Treatment Analytics System

Project Report

Submitted by

SAMEER JAKKUVA — 24BDS026
ABHANG PAWAR — 24BDS054
PIYUSH PRASHANT — 24BDS055
PRIYANSHU MITTAL — 24BDS058

Under the guidance of

Dr. Utkarsh Khaire

Department of Data Science and Artificial Intelligence
Indian Institute of Information Technology Dharwad

Academic Year: 2025–2026

Abstract

Healthcare administration demands sophisticated data management to ensure operational efficiency, proper resource allocation, and optimal patient outcomes. The CareOps project delivers a comprehensive, end-to-end Database Management System (DBMS) and Data Warehousing architecture designed to model hospital operations and provide decision-oriented analytics.

The architecture is partitioned into two environments. The first is an Online Transaction Processing (OLTP) database containing 11 fully normalised (3NF) tables tracking patients, doctors, wards, visits, diagnoses, and inpatient bed allocations, augmented with triggers, stored procedures, and operational views. A Python simulation (data.py) populates the schema with 500+ patients and 2,000+ hospital visits using authentic ICD-10 classifications. The second tier is an Online Analytical Processing (OLAP) Star-Schema Data Warehouse fed by a Python ETL pipeline (etl_code.py). Using advanced SQL analytics — temporal windowing, CTEs, and aggregation — CareOps delivers five granular business intelligence reports covering resource consumption, clinical recovery, ward utilisation, financial volatility, and readmission risk. A full-stack React + Express.js dashboard renders all five reports as interactive browser charts.

1 Introduction

1.1 Background and Motivation

Modern hospitals are both clinical care hubs and massive data engines. Every patient interaction generates highly relational data — from demographic registries and pathological diagnostics to pharmaceutical billing and real-time bed tracking. Foundational OLTP databases manage these transactions effectively, but are architecturally ill-suited to longitudinal analytics. Running a 3-month moving average of treatment costs for Myocardial Infarction patients on a live operational database introduces query contention that directly degrades admission workflows.

CareOps addresses this bottleneck with a bifurcated professional-grade analytics pipeline: an OLTP source mapped via an ETL pipeline to an isolated Data Warehouse, giving administrators unhindered access to historical intelligence without touching live transactional systems.

1.2 Business Objectives

The data warehouse architecture is engineered to answer five strategic business questions:

- Resource Depletion Analysis: Which diseases consume the highest resources in treatment cost and ward stay duration?
- Clinical Outcome Efficacy: Which doctors record the strongest recovery outcomes across a statistically valid patient volume?
- Infrastructure Over-utilisation: Which wards show sustained periods of high congestion or critical capacity risk?
- Financial Volatility Analysis: How are treatment costs trending month-over-month across clinical disciplines?
- Readmission Penalties: Which diseases represent the highest probability of cyclic patient readmission?

1.3 Technology Stack

Layer	Technology
Database	MySQL 8.x
ETL and data simulation	Python 3.10+
Python DB connector	mysql-connector-python
Analytics	SQL — CTEs, window functions, RANK, LAG
Dashboard front-end	React + Vite
Dashboard back-end	Express.js + mysql2

Table 1.1 — CareOps Technology Stack

2 System Requirements and Specifications

2.1 Functional Requirements

The CareOps relational layer satisfies the following primary directives:

- FR-01: Store anonymised patient demographics — age bands, gender, blood group, city.
- FR-02: Record chronological patient visits bound to an assigned physician.
- FR-03: Document diagnoses mapped via ICD-10 codes with algorithmic severity markers.
- FR-04: Itemise clinical treatments with pharmaceutical dosages, costs, and duration.
- FR-05: Govern ward bed utilisation, logging admit and discharge intervals precisely.
- FR-06: Classify treatment outcomes as Recovered, Improved, Readmitted, or Deceased.

2.2 Non-Functional Requirements and Scale

The schema complies fully with Third Normal Form (3NF), enforcing data atomicity and preventing update anomalies. The system is validated against production-grade test loads: 500+ patients, 50 physicians, 30 ICD-10 disease classifications, and approximately 2,000 synthesised visit interactions. Automated consistency is monitored via embedded DBMS triggers.

2.3 Project Phase Summary

Phase	Deliverable	Status
Phase 0	Scope definition	Complete
Phase 1	Requirements specification	Complete
Phase 2	OLTP schema — 11 MySQL tables	Complete
Phase 3	Trigger, stored procedure, and view	Complete
Phase 4	OLTP data simulation (data.py)	Complete
Phase 5	Data warehouse schema and dimension load	Complete
Phase 6	ETL pipeline (etl_code.py)	Complete
Phase 7	Analytics SQL queries and reporting	Complete
Phase 8	React + Express.js dashboard	Implemented

Table 2.1 — Phase-by-Phase Project Status

3 Operational Database Design (OLTP)

The CareOps live environment (careops_oltp) comprises eleven interlocked MySQL 8.x tables enforcing referential integrity via foreign keys. Tables are grouped into primary master entities (no foreign dependencies) and transactional/relational entities that bind the master records into medical engagements.

3.1 Primary Master Entities

The five master tables form the unencumbered foundation of the hospital ecosystem:

Table	Primary Key	Key Attributes / Constraints
Patient	patient_id (INT, AUTO)	age_group VARCHAR(20), gender CHAR(1) CHECK IN ('M','F','O'), blood_group VARCHAR(5), city VARCHAR(50)
Doctor	doctor_id (INT, AUTO)	specialization VARCHAR(80) NOT NULL, experience_years INT CHECK ≥ 0, department VARCHAR(60)
Disease	disease_code VARCHAR(10)	disease_name NOT NULL, category NOT NULL, severity_tier CHECK IN ('Low','Moderate','High','Critical')
Medicine	medicine_id (INT, AUTO)	medicine_name NOT NULL, medicine_type VARCHAR(50), unit_cost DECIMAL(10,2) CHECK ≥ 0
Ward	ward_id (INT, AUTO)	ward_name NOT NULL, ward_type NOT NULL, capacity INT CHECK > 0

Table 3.1 — Primary Master Entities

3.2 Transactional and Relational Entities

Six transactional tables bind master attributes into definitive medical engagements:

Table	Key / Constraints	Notable Design Decision
Visit	PK: visit_id; FK: patient_id, doctor_id	visit_type bounded to 'OPD', 'IPD', 'Emergency'
Diagnosis	Composite PK: (visit_id, disease_code)	Prevents identical disease logged twice per visit — zero redundancy
Treatment	PK: treatment_id; FK: visit_id, medicine_id	dosage_mg DECIMAL(8,2); duration_days CHECK > 0
BedAllocation	PK: allocation_id; FK: patient_id, ward_id	days_stayed GENERATED ALWAYS AS DATEDIFF(discharge_date, admit_date) — eliminates miscalculation
Outcome	PK: outcome_id; UNIQUE: visit_id	status CHECK IN ('Recovered','Improved','Readmitted','Deceased') — one outcome per visit
AlertLog	PK: alert_id	Populated exclusively by trg_long_stay trigger — never written manually

Table 3.2 — Transactional and Relational Entities

4 Advanced DBMS Functionalities

Phase 3 elevates CareOps from a passive data repository to an active DBMS by embedding logic directly on the database server. Three programmable artefacts implement automated auditing, transactional safety, and real-time executive reporting.

4.1 Trigger — trg_long_stay

An AFTER UPDATE trigger on BedAllocation monitors patient IPD release metrics. When days_stayed exceeds 14 consecutive days, the trigger automatically inserts an alert record into the AlertLog table, creating a transparent liability-oversight channel for medical superintendents without any application-tier involvement.

```
CREATE TRIGGER trg_long_stay
AFTER UPDATE ON BedAllocation FOR EACH ROW
BEGIN
  IF NEW.days_stayed IS NOT NULL AND NEW.days_stayed > 14 THEN
    INSERT INTO AlertLog (patient_id, ward_id, days_stayed, alert_message)
    VALUES (NEW.patient_id, NEW.ward_id, NEW.days_stayed,
      CONCAT('Patient ', NEW.patient_id, ' stayed ', NEW.days_stayed,
             ' days in ward ', NEW.ward_id, '. Review required.'));
  END IF;
END
```

4.2 Stored Procedure — admit_patient

The admit_patient procedure enforces capacity-aware admission. Before creating a Visit or BedAllocation record, it queries active occupancy (discharge_date IS NULL) and compares it against the ward's physical capacity. If the ward is full, it raises SQLSTATE '45000' — a hard SQL exception preventing double-occupancy without any application-layer guard.

```
-- Core capacity check logic
SELECT capacity INTO ward_cap FROM Ward WHERE ward_id = p_ward_id;
SELECT COUNT(*) INTO current_occ FROM BedAllocation
  WHERE ward_id = p_ward_id AND discharge_date IS NULL;

IF current_occ >= ward_cap THEN
  SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT = 'Ward at full capacity. Cannot admit patient.';
ELSE
  INSERT INTO Visit ... ; INSERT INTO BedAllocation ...;
END IF;
```

4.3 View — vw_doctor_performance

A structured SQL view unifies Visits, BedAllocations, and Outcomes via LEFT JOINs to construct aggregate performance arrays per physician. It computes total_visits, avg_patient_stay, recovered_count, readmission_count, and recovery_rate_pct — serving as a live, fully abstracted dashboard pipeline for executives without requiring ad-hoc queries.

5 Data Warehousing and ETL Pipeline

5.1 OLTP Data Simulation

Testing an analytical pipeline without realistic data yields false confidence. Phase 4 employs a Python generator (data.py) seeded with random.seed(42) for full reproducibility. The generator uses categorical distribution weightings rather than pure randomness to produce clinically authentic outcomes:

- 50 physicians across 10 specialisations (Cardiologist, Neurologist, Oncologist, Nephrologist, and others)
- 30 WHO ICD-10 codes — critical classifications like A41.9 Sepsis purposefully produce longer ward stays than localised ailments like A09 Gastroenteritis
- 60+ therapeutics with historically accurate unit costs (e.g. Ceftriaxone, Atorvastatin, Ibuprofen)
- Outcome weights: 55% Recovery, 30% Improved, 12% Readmission, remainder Deceased — representing realistic chronic failure rates

Execution yields ~10,000 interlocking cross-table DML iterations, populating the CareOps schema across simulated years 2022–2023.

5.2 Star Schema Data Warehouse

To answer strategic KPIs, CareOps establishes a dedicated careops_dw schema organised under Dimensional Modeling (Star Schema). Highly normalised OLTP databases require expensive cascading joins for summary queries; dimensional modelling denormalises these into predictable, sub-second lookups.

Dimension Tables

Dimension	Surrogate Key	Key Attributes
Dim_Date	date_id (YYYYMMDD INT)	full_date, day_of_week, month_num, quarter, year_num
Dim_Disease	disease_sk (AUTO)	disease_code, disease_name, category, severity_tier
Dim_Doctor	doctor_sk (AUTO)	doctor_id, specialization, experience_band, department
Dim_Ward	ward_sk (AUTO)	ward_id, ward_name, ward_type, capacity
Dim_Outcome	outcome_sk (AUTO)	outcome_label (Recovered / Improved / Readmitted / Deceased)

Table 5.1 — Data Warehouse Dimension Tables

Dim_Date pre-decomposes visit dates into calendar attributes (year_num, month_num, quarter), eliminating repeated DATE_FORMAT calls in analytical queries. Dim_Doctor pre-computes experience_band buckets ('0-5 yrs', '6-10 yrs', '11+ yrs') for cohort-based performance groupings.

Fact Table — Fact_Treatment

The computational epicenter receives surrogate keys to all five dimensions alongside quantitative measures. Millions of transactional rows are aggregated into thousands of measurable fact rows, enabling sub-second KPI queries.

Column	Type	Description
fact_id	BIGINT PK AUTO	Unique fact row identifier
date_id / disease_sk / doctor_sk	INT FK	References to all five dimension tables
ward_sk / outcome_sk	INT FK (nullable)	Ward and outcome dimension references
total_cases / recovered_cases / readmitted_cases	INT NOT NULL	Aggregate case counts per group
avg_recovery_days / avg_treatment_cost	DECIMAL	Mean per-case recovery duration and cost
total_bed_days	INT	Aggregate inpatient bed-days consumed

Table 5.2 — Fact_Treatment Structure

5.3 ETL Pipeline (etl_code.py)

Phase 6 bridges the two environments via a Python ETL application maintaining simultaneous connections to both careops_oltp and careops_dw:

- **Extract:** A multi-JOIN query bridges Visit, Diagnosis, Outcome, and BedAllocation, producing a wide flat record for every historical visit.
- **Transform:** Python collections.defaultdict arrays accumulate rows by composite identity (Date + Disease + Ward), incrementally computing total_cases, average treatment costs, and bed-day sums.
- **Load:** Natural keys (e.g. the string disease code J18.9) are replaced with robust numeric surrogate keys from the dimension tables before bulk-loading via executemany parameterisation.

6 Business Analytics and Interpretations

All five Phase 7 queries operate on careops_dw and support optional date-range filtering via @from_date and @to_date session variables (analytics_queries.sql).

6.1 Q1 — Disease Resource Consumption Trend

CTEs merge chronological filters with financial facts to highlight resource-draining illnesses. A weighted avg_treatment_cost × total_cases multiplier produces aggregate expenditure, while a 3-month moving average smooths temporal noise:

```
ROUND(AVG(total_treatment_spend) OVER (
  PARTITION BY disease_name ORDER BY year_num, month_num
  ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
), 2) AS spend_3m_moving_avg
```

Rank	Period	Disease	Cases	Bed Days	Total Spend ($)
1	2023-08	Sepsis	8	104	1,262,236
2	2022-12	Viral Infection	4	74	541,554
3	2023-08	Hypertension	15	146	489,962
4	2023-09	Hip Osteoarthritis	9	89	485,024
5	2023-07	Acute Myocardial Infarction	6	55	456,882

Table 6.1 — Top Medical Expenditure Drivers

High-spend spikes concentrate around discrete severe incidents rather than high-volume infections. Treatment pathways for Sepsis and Hypertension clusters warrant aggressive administrative audit to limit systemic cost overruns.

6.2 Q2 — Doctor Recovery Performance Ranking

Rankings are restricted to physicians with N ≥ 30 cases for statistical validity. A composite RANK() orders by recovery rate descending, readmission rate ascending, and case volume descending.

Doc. ID	Specialisation	Cases	Recovery %	Readmit %
49	Endocrinologist	63	77.78	15.87
9	Endocrinologist	77	71.43	7.79
15	Dermatologist	51	66.67	1.96
24	Orthopedic Surgeon	37	64.86	8.11
30	Nephrologist	82	64.63	7.32

Table 6.2 — Empirical Physician Outcome Ranking

Pronounced operational variances exist across departments. Doctors 49 and 9 should serve as standardisation mentors to port best-practice clinical protocols into lower-performing sectors.

6.3 Q3 — Ward Over-Utilisation Analysis

Monthly utilisation is computed as (occupied_bed_days ÷ capacity × days_in_month) × 100, then classified into bands: Critical Over-Utilisation ≥ 90%, High ≥ 75%, Moderate ≥ 50%, Low < 50%.

Period	Ward	Type	Capacity	Occupied Bed Days	Util. %	Band
2023-08	ICU	ICU	10	247	79.68	High
2023-08	Private Ward	Private	10	236	76.13	High
2023-02	Cardiac ICU	ICU	8	168	75.00	High

Period	Ward	Type	Capacity	Occupied Bed Days	Util. %	Band
2023-07	Cardiac ICU	ICU	8	186	75.00	High
2023-11	Cardiac ICU	ICU	8	177	73.75	Moderate

Table 6.3 — Infrastructure Traffic Hotspots

Intensive care environments routinely breach the 75% high-utilisation threshold. Capital investment should prioritise auxiliary Cardiac ICU modules immediately.

6.4 Q4 — Treatment Cost Volatility

SQL LAG() window functions expose precise month-over-month variance ratios across hospital expenditure operations.

Year-Month	Avg Treatment Cost ($)	MoM Change %
2023-12	13,876.77	−5.96
2023-11	14,755.47	−1.73
2023-10	15,015.36	−1.14
2023-09	15,188.36	−5.17
2023-08	16,016.91	−13.39
2023-07	18,492.74	+63.63

Table 6.4 — Sequential Cost Trend Volatilities

A structural +63.63% deviation in July 2023 is clearly isolated. Future reporting alerts should determine whether this reflects pathological supply-chain volatility or an external systemic shock.

6.5 Q5 — Chronic Readmission Burden

Diseases with high readmission percentages reveal failed discharge protocols. A minimum threshold of N ≥ 25 cases suppresses small-sample noise before ranking.

Rank	Disease	Cases	Readmitted	Readmission %	Recovery %
1	GI Bleeding	126	23	18.25	48.41
2	Urinary Tract Infection	117	20	17.09	52.14
3	Acute Pharyngitis	113	19	16.81	55.75
4	Kidney Stones	100	16	16.00	56.00
5	Epilepsy	122	18	14.75	54.10

Table 6.5 — Recalcitrant Medical Classifications

Systemic breakdown is visible in managing GI Bleeding and UTI, both with ~18% recurrence rates. Tighter post-discharge pharmaceutical compliance pathways are essential to break readmission cycles.

7 Interactive Web Dashboard

Phase 8 operationalises all five analytical KPIs as browser-accessible interactive charts. The full-stack application consists of a React 18 + Vite front-end and an Express.js API server using mysql2, with CORS-enabled JSON communication on port 4000.

7.1 API Endpoints and Dashboard Pages

API Route	React Page	Content
GET /api/meta	DashboardPage.jsx	Last fact date, total fact rows, summary KPIs
GET /api/disease	DiseasePage.jsx	Q1 disease consumption trend with chart
GET /api/doctors	DoctorPage.jsx	Q2 physician performance ranking — bar chart
GET /api/wards	WardPage.jsx	Q3 ward utilisation — heatmap-style chart
GET /api/readmissions	ReadmissionPage.jsx	Q5 readmission burden — ranked chart

Table 7.1 — Phase 8 API Endpoints

All endpoints accept optional ?from=YYYY-MM-DD&to=YYYY-MM-DD query parameters, forwarded to underlying SQL date-range filters for interactive time-window selection.

8 Conclusion and Strategic Outlook

8.1 Summary

CareOps represents a complete implementation of modern data warehousing strategies applied to a realistic medical infrastructure. By strictly separating OLTP transactional queries from OLAP historical analytical sweeps, the system protects live clinical environments from SQL-locking failures while giving management ubiquitous access to historical intelligence.

The ETL pipeline converts millions of disjoint real-time records into pristine numerical metrics inside a dimensional Star Schema. Phase 7 analyses give management the tools to preemptively curtail ICU flooding, isolate expenditure spikes, and identify systemic performance lapses. Phase 8 closes the loop between raw data and actionable executive reporting through a browser-accessible dashboard.

8.2 Key Technical Achievements

- Fully normalised 11-table OLTP schema in MySQL 8.x with enforced referential integrity and CHECK constraints
- Active DBMS logic: long-stay trigger, capacity-aware admission procedure, and a live performance summary view
- Reproducible Python simulation — 500+ patients, 50 physicians, 2,000+ visits with ICD-10 classifications
- Star Schema data warehouse with 5 dimension tables and a Fact_Treatment table bridged via a Python ETL
- Five production-grade SQL analytical queries using CTEs, window functions (LAG, RANK, AVG OVER), and NULLIF safeguards
- Full-stack React + Express.js dashboard consuming live data warehouse aggregates

8.3 Recommended Strategic Actions

1. Establish disease-specific readmission-prevention bundles for GI Bleeding, UTI, and Acute Pharyngitis.
2. Review ICU and Cardiac ICU capacity plans for months exceeding 75% utilisation.
3. Launch a cost-variance investigation focused on the July 2023 expenditure spike.
4. Build clinician peer-learning reviews drawing from top-ranked performers — Doctor IDs 49 and 9.
5. Adopt the five Phase 7 queries as a fixed monthly KPI governance baseline.

References

[1] Kimball, R., & Ross, M. (2013). The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling (3rd ed.). Wiley.

[2] Inmon, W. H. (2005). Building the Data Warehouse (4th ed.). Wiley.

[3] Oracle Corporation. (2025). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

[4] Python Software Foundation. Python Database API Specification 2.0 (PEP 249). https://peps.python.org/pep-0249/

[5] World Health Organization. International Statistical Classification of Diseases (ICD-10). https://icd.who.int/browse10/2019/en

[6] Meta Platforms, Inc. (2024). React Documentation. https://react.dev/

[7] OpenJS Foundation. (2024). Express.js Documentation. https://expressjs.com/

Appendix A — System Setup and Execution Guide

A.1 Prerequisites

- MySQL 8.x running locally
- Python 3.10+ with packages: mysql-connector-python, faker
- Node.js 18+ (Phase 8 dashboard only)

A.2 Environment Configuration

Create a .env file at the project root:

```
CAREOPS_DB_HOST=localhost
CAREOPS_DB_PORT=3306
CAREOPS_DB_USER=root
CAREOPS_DB_PASSWORD=password
CAREOPS_OLTP_DB=careops_oltp
CAREOPS_DW_DB=careops_dw
```

A.3 Automated Run (Windows PowerShell)

```
./run_demo.ps1
```

A.4 Manual Phase-by-Phase Execution

```
# Phase 2: Create OLTP schema
mysql -u root -p < phase2_oltp/oltp_table.sql

# Phase 3: Apply trigger, procedure, and view
mysql -u root -p careops_oltp < phase3_adv/phase3_all.sql

# Phase 4: Generate synthetic data
python phase4_genData/data.py

# Phase 5: Create DW schema and populate dimensions
mysql -u root -p < phase5_datawarehouse/createDW.sql
python phase5_datawarehouse/addData.py

# Phase 6: Run ETL pipeline
python phase6_etl/etl_code.py

# Phase 7: Execute analytics queries
mysql -u root -p careops_dw < phase7_analytics/analytics_queries.sql
```

A.5 Post-Run Verification

```
-- OLTP load check
SELECT COUNT(*) AS patient_count FROM careops_oltp.Patient;
SELECT COUNT(*) AS visit_count   FROM careops_oltp.Visit;

-- DW load check
SELECT COUNT(*) AS fact_count FROM careops_dw.Fact_Treatment;

-- Monthly case distribution
SELECT dd.year_num, dd.month_num, SUM(ft.total_cases) AS total_cases
FROM careops_dw.Fact_Treatment ft
JOIN careops_dw.Dim_Date dd ON dd.date_id = ft.date_id
GROUP BY dd.year_num, dd.month_num ORDER BY 1, 2;
```